

Final Exam — Sample

Name: _____

Software Engineering — UATX — Spring 2026

Format	Pencil and paper. No computer, phone, notes, or coding agent.
Time	150 minutes.
Length	Seven questions, one to two per category.
Coverage	Lectures 5.1 through 10.1 (JS/TS and the browser, React, deployment, Docker, CI/CD, concurrency, security, agents), plus assignments 3 and 4.
Categories	Trace-through, spot the bug, specification, code review, short design.
Grading	Curved. The exam is intentionally hard; expect to leave some parts unfinished.

HOW THIS EXAM THINKS ABOUT AGENTS

The whole semester has leaned into coding agents as the way real engineers work, so an exam that asks you to write working code from scratch on paper would punish you for taking the course seriously. Instead, every question is built around a skill the agent can't do for you: reading code carefully, finding bugs by inspection, writing the spec you'd hand to the agent, judging which of two implementations is better, and making design calls under uncertainty.

HOW TO USE THIS SAMPLE

This is a representative exam, not the actual one -- the real final will have seven questions in the same five categories at roughly the same difficulty. Solutions are in a separate document; try the questions cold first, then check.

Question 1

Trace-through

The script runs to completion. Give the value printed at A, B, C, and D.

```
const todos = [
  { id: 1, title: "read notes", done: true, priority: 2 },
  { id: 2, title: "write fetch", done: false, priority: 1 },
  { id: 3, title: "pick up milk", done: false, priority: 3 },
  { id: 4, title: "call the vet", done: true, priority: 1 },
];

const a = todos.filter(t => !t.done).map(t => t.title);
console.log("A:", a); // ____

const b = todos.filter(t => t.done).length;
console.log("B:", b); // ____

const c = todos.filter(t => !t.done).reduce((s, t) => s + t.priority, 0);
console.log("C:", c); // ____

console.log("D:", 1 === "1", 0 == false, null === undefined); // ____
```

Question 2

Trace-through

A promo code grants \$10 of account credit, once per user. On a flaky connection a user double-taps redeem, so two identical requests arrive at nearly the same instant. The handler is below; both requests run their SELECT before either runs its INSERT.

```
def redeem(user_id, code):
    already = db.execute(
        text("SELECT 1 FROM redemptions WHERE user_id = :u AND code = :c"),
        {"u": user_id, "c": code},
    ).first()
    if already:
        raise HTTPException(409, "already redeemed")
    db.execute(text("INSERT INTO redemptions (user_id, code) VALUES (:u, :c)"),
               {"u": user_id, "c": code})
    db.execute(text("UPDATE accounts SET credit = credit + 10 WHERE user_id = :u"),
               {"u": user_id})
```

1. What happens, and how much credit does the user end up with?
2. A teammate wraps the whole handler in one transaction (with `db.engine.begin()` as `conn:` for each request). Does that fix the issue? Explain.
3. What actually prevents it? Give one concrete mechanism and why it is atomic.

Question 3

Spot the bug

Your agent produced this submit handler to POST a new message and show it. It works when you click slowly on a good connection. Find at least three defects; for each, one sentence on what goes wrong and one on the fix.

```
const form = document.querySelector("#new") as HTMLFormElement;
const input = document.querySelector("#text") as HTMLInputElement;
const list = document.querySelector("#list") as HTMLUListElement;

form.addEventListener("submit", async (event) => {
  const response = await fetch("/api/messages", {
    method: "POST",
    body: { text: input.value },
  });
  const created = await response.json();
  const li = document.createElement("li");
  li.textContent = input.value;
  list.append(li);
});
```

Question 4

Spot the bug

Your agent produced this endpoint for a project-management app (`TaskUpdate` is a Pydantic model). It parameterizes its values and it checks authorization. Read it carefully.

```
@app.patch("/api/projects/{project_id}/tasks/{task_id}")
def update_task(project_id: int, task_id: int, body: TaskUpdate,
                user: User = Depends(current_user)):
    with engine.begin() as conn:
        member = conn.execute(
            text("SELECT 1 FROM project_members "
                "WHERE project_id = :p AND user_id = :u"),
            {"p": project_id, "u": user.id},
        ).first()
        if member is None:
            raise HTTPException(403, "not a member of this project")

        fields = body.model_dump(exclude_unset=True)
        if not fields:
            raise HTTPException(400, "nothing to update")
        set_clause = ", ".join(f"{col} = :{col}" for col in fields)
        result = conn.execute(
            text(f"UPDATE tasks SET {set_clause} WHERE id = :id"),
            {**fields, "id": task_id},
        )
        if result.rowcount == 0:
            raise HTTPException(404)
        return {"ok": True}
```

1. Name two things this endpoint does well, security-wise.
2. It has a serious access-control hole. Describe it, give a concrete request that exploits it, and give the fix.
3. The `SET` clause is built from whatever fields are in the request body. Explain the risk that creates, and what `TaskUpdate` has to guarantee for it to be safe.

Question 5

Specification

Feature request to your agent: "As the user types in the search box, show matching products." The agent's first cut is below, and it works in a quick demo. Before you let it ship, write the spec you would hand back. Do not write code.

```
function ProductSearch() {
  const [results, setResults] = useState([]);
  const [query, setQuery] = useState("");
  useEffect(() => {
    fetch(`/api/products?q=${query}`)
      .then(r => r.json())
      .then(setResults);
  }, [query]);
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      <ul>{results.map(p => <li key={p.id}>{p.name} (${p.price})</li>)}</ul>
    </div>
  );
}
```

1. The display states this UI must handle -- name them.
2. Two behaviors it is currently missing or getting wrong (look hard at a fast typist on a flaky network).
3. One product decision that is ambiguous, and the call you would make.

Question 6

Code review

This workflow runs the test suite on every push; two of the tests are shown below. On the developer's laptop the suite passes every time. In CI, `test_lists_single_user` fails about half the time.

```
# .github/workflows/test.yml
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with: { python-version: "3.14" }
      - run: pip install -r requirements.txt
      - run: pytest -n auto          # runs tests in parallel across CPUs
```

```
# conftest.py + test_users.py (app talks to the module-level db.engine)
@pytest.fixture
def client():
    return TestClient(app)

def test_create_user(client):
    r = client.post("/users", json={"username": "alice"})
    assert r.status_code == 201

def test_lists_single_user(client):
    users = client.get("/users").json()
    assert len(users) == 1
```

1. Why does `test_lists_single_user` pass locally but fail intermittently in CI?
2. Is the bug in the workflow or in the tests? Point to the specific cause.
3. Give two fixes -- one to the tests, one you could make to the workflow -- and say which is the real fix.

Question 7

Short design

Your agent generated this Dockerfile for the FastAPI service. It is the team's deploy image: pushed to a shared registry (GitHub Container Registry) that teammates and the deploy server pull from, and rebuilt on every push to `main`. The project directory also contains a `.env` with more secrets, and there is no `.dockerignore`.

```
FROM python:3.14
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# image-processing system library the app shells out to
RUN apt-get update && apt-get install -y imagemagick

ENV JWT_SECRET="prod-signing-key-7f3a9c2b"
ENV DATABASE_URL="postgresql://app:s3cr3t@db.internal:5432/app"

CMD uvicorn app:app --host 0.0.0.0 --port $PORT
```

1. The team rebuilds this image many times a day as they edit application code, and rebuilds are slower than they should be. One thing here is already done right and one step is the culprit -- identify both, and restructure so a one-line change to `app.py` rebuilds quickly.
2. This image is pushed to a shared registry. What secrets does it expose, by what mechanisms, and how do you keep them out of the image?
3. You realize this image was already pushed last week. You delete the secrets from the Dockerfile and add a `.dockerignore` now. Is that enough? What do you actually do?

